Práctica de Aprendizaje Supervisado

Resumen rápido · Dataset Wireless Indoor Localization (discretizado) · Árbol de decisión y Naive Bayes

1. El dataset en 30 segundos

Cada fila es una medición: alguien estaba con el móvil en una habitación y se apuntó cuánta señal WiFi recibía de 7 routers. El modelo aprende a hacer lo contrario: le das las 7 señales y te dice en qué habitación estás. Es un problema de clasificación.

Columna	Qué es	Valores
WS1 … WS7	Intensidad de la señal de cada router (discretizada)	Enteros pequeños, de -1 a -10
Room	Habitación donde estaba el móvil (columna objetivo)	1, 2, 3 o 4

2000 filas, 500 por habitación (25 % cada una) → dataset equilibrado. Fichero: `wifi_localization_discretizado.csv` (sin cabecera, separado por comas).

2. ¿Qué significan los números? (la discretización)

La señal original se mide en dBm (de -10 a -98) y tenía demasiados valores posibles. Para que se parezca al ejemplo de los coches (pocos valores por atributo), se dividió entre 10 y se redondeó: -64 → -6, -68 → -7, -38 → -4. Cuanto más cerca de 0, más fuerte es la señal. Room no es fuerza de señal: es solo la etiqueta de la habitación.

Atributo	Posibles valores
WS1	-1 a -7 (7 valores)
WS2	-5, -6, -7
WS3	-4 a -7
WS4	-1 a -8
WS5	-4 a -9
WS6	-6 a -10
WS7	-6 a -10

3. Qué pedía la práctica y cómo está hecho

Punto	Cómo lo hemos resuelto	Estado
1. Elegir CSV y explicarlo	Celdas de texto con contenido, atributos (WS1–WS7), discretización y objetivo (Room)	✔
2. Instalar paquetes	`%pip install pandas numpy scikit-learn` (en CMD: `pip install ...`)	✔
3. Entrenar 66/33 estratificado	`train_test_split(test_size=.33, stratify=...)` → 1340 entrenamiento / 660 prueba, 25 % por habitación en ambos	✔
4. Pruebas con datos externos	Mediciones inventadas (escala -1…-10) → el modelo acierta la habitación	✔

4. Resultados

Modelo	Acierto (accuracy) sobre 660 pruebas
Árbol de decisión (ID3, entropía)	97,27 %
Naive Bayes (CategoricalNB)	97,12 %

Informe del árbol por habitación:

Habitación	Precision	Recall	Support
1	0.98	0.99	165
2	1.00	0.96	165
3	0.95	0.97	165
4	0.96	0.97	165

- Precision: cuando dice «habitación X», cuántas veces acierta.
- Recall: de las mediciones que eran de verdad de X, cuántas encuentra.
- Support: casos de prueba reales de cada habitación. 165 × 4 = 660 (el 33 % de 2000). Que sean iguales demuestra que la estratificación funciona.
- accuracy / macro avg / weighted avg: no son habitaciones, son resúmenes de todas. Macro y weighted coinciden porque las clases están equilibradas.

5. Pasos del cuaderno (igual que el ejemplo de los coches)

- Leer el CSV:

```
wifi = pandas.read_csv('wifi_localization_discretizado.csv', header=None,
        names=['WS1','WS2','WS3','WS4','WS5','WS6','WS7','Room'])
```

- OrdinalEncoder para los atributos: convierte -7, -6, -5… en 0, 1, 2… (CategoricalNB no acepta negativos).
- LabelEncoder para Room: 1–4 → 0–3.
- train_test_split 66/33 estratificado con `random_state=12345`.
- Árbol: en las reglas sale `WS1 <= 1.50` porque usa los códigos del encoder, no los valores reales. Y `class: 0` = habitación 1, class 1 = hab. 2, etc.
- Datos externos: DataFrame con WS1–WS7 → `codificador_atributos.transform(...)` → `predict`. Ojo: solo valores que existan en el fichero, si no el encoder da error.

6. Cómo es cada habitación (valor más habitual)

Hab.	WS1	WS2	WS3	WS4	WS5	WS6	WS7	Lo que la distingue
1	-6	-6	-6	-6	-7	-8	-8	Señal media en casi todo
2	-4	-6	-6	-4	-7	-7	-7	Muy fuerte en routers 1 y 4
3	-5	-6	-5	-5	-6	-8	-8	Intermedia en routers 1–4
4	-6	-6	-5	-6	-5	-9	-9	Fuerte en el router 5

7. Antes de exponer

- Kernel → Restart & Run All: todo debe ejecutarse de arriba abajo sin errores.
- Si sale `NameError`, es que una celda anterior no se ha ejecutado: ejecutar siempre en orden.